

FoodXpress Web Application

Complete Web Controllers Architecture & Code Explanation Guide

Executive Summary & MVC Architecture Overview:

In the **FoodXpress** Java EE web application, **Controllers (Servlets)** act as the central request handlers and traffic orchestrators. Extending `HttpServlet`, these 17 controllers receive HTTP GET and POST requests from the client browser, interact with the MySQL database via the **Data Access Object (DAO)** layer, update HTTP Session and Request scopes, and forward user control to JSP view templates.

This document provides a comprehensive technical breakdown of all **17 Controller Servlets** in the system, combining high-level architectural responsibilities, URL mappings, HTTP method flows, and exact Java source code.

■ Controller Summary Matrix (17 Servlets)

#	Servlet Class	URL Pattern	Role / Access	Primary Function
1	UserServlet	/user	Public / All Users	User Authentication, Account Registration, and Session Management Controller.
2	RestaurantServlet	/restaurant	Customer / Public	Restaurant Listing, Search, and Detailed Profile Discovery Controller.
3	CartServlet	/cart	Customer	Shopping Cart Session State & Price Calculation Engine.
4	CheckoutServlet	/checkout	Customer	Order Placement, Address Assignment, and Transaction Persistence Controller.
5	OrderHistoryServlet	/orderHistory	Customer	Customer Order History Timeline & Tracking Controller.
6	ReviewServlet	/review	Customer	Customer Rating (1-5 Stars) & Feedback Submission Controller.
7	AddressServlet	/address	Customer	Customer Delivery Address CRUD Controller.
8	AdminDashboardServlet	/admin/dashboa rd	Admin Only	Administrator Analytics & Platform Summary Controller.
9	AdminOrderServlet	/admin/orders	Admin Only	Administrator Order Dispatching & Live Status Management Controller.
10	AdminUserServlet	/admin/users	Admin Only	User Account Management & Role Elevation Controller.
11	AdminReviewServlet	/admin/reviews	Admin Only	Customer Feedback & Review Moderation Controller.
12	AdminRestaurantServlet	/admin/restaura nts	Admin Only	Bangladeshi Restaurant CRUD Management Controller.
13	AdminMenuServlet	/admin/menu	Admin Only	Restaurant Menu Items & Dish Pricing (■) CRUD Controller.
14	MenuServlet	/menu	Public / Customer	Menu Item Query & Search Controller.
15	OrderSummaryServlet	/orderSummary	Customer	Pre-Checkout Itemized Invoice Review Controller.
16	PaymentServlet	/payment	Customer	Payment Processing & Gateway Simulation Controller.
17	OrderSuccessServlet	/orderSuccess	Customer	Order Confirmation & Final Receipt Controller.

■ Deep-Dive Controller Analysis & Source Code

1. UserServicelet.java (URL: /user)

Role / Access Level: Public / All Users

Primary Purpose: User Authentication, Account Registration, and Session Management Controller.

Architectural Overview: UserServicelet is the primary gateway for identity management in FoodXpress. It handles user registration, password hashing via BCrypt, credential validation upon login, session initialization (storing user entity and assigned role), and secure session destruction upon logout.

HTTP Methods & Flow Breakdown:

- **doGet():** Processes logout requests by invalidating the HTTP Session and forwarding users to login/home pages.
- **doPost():** Handles form submissions for login (`action=login`) and registration (`action=register`). Checks credentials, hashes passwords, sets session attributes (`loggedUser`, `userRole`), and executes role-based redirection.

Key Implementation Highlights:

- **Password Security:** Utilizes BCrypt hashing before persisting user records into MySQL via `UserDAO`.
- **Role-Based Routing:** If an authenticated user's role is 'admin', they are automatically redirected to `/admin/dashboard`. Customers are forwarded to `/jsp/customer/home.jsp`.
- **Session Management:** Stores user state in `HttpSession.setAttribute("loggedUser", user)` for global session persistence across requests.

Source Code (UserServicelet.java):

```
1 package com.app.controllers;
2
3 import java.io.IOException;
4 import javax.servlet.RequestDispatcher;
5 import javax.servlet.ServletException;
6 import javax.servlet.annotation.WebServlet;
7 import javax.servlet.http.*;
8
9 import com.app.dao.UserDAO;
10 import com.app.dao_implementation.UserDAOImpl;
11 import com.app.models.User;
12 import org.mindrot.jbcrypt.BCrypt;
13
14 // This servlet handles all user-related actions
15 /*This UserServicelet controls:
16
17 ✓ Registration
18 ✓ Login
19 ✓ Logout
20 ✓ Showing profile
21 ✓ Updating profile
22 ✓ Redirecting to JSP pages
23 * */
24 @WebServlet("/user") // Maps this servlet to /user URL
25 public class UserServicelet extends HttpServlet {
26     private static final long serialVersionUID = 1L;
27
28     // DAO object for interacting with the database
29     private UserDAO userDao;
30
31     @Override
32     public void init() throws ServletException {
33
34         // Initialize DAO object. Called once when servlet loads.
35         userDao = new UserDAOImpl();
```

```
36     }
37
38     // -----
39     // GET Requests
40     // -----
41     protected void doGet(HttpServletRequest request, HttpServletResponse response)
42         throws ServletException, IOException {
43
44         // Determine what action is requested (default = login)
45         String action = request.getParameter("action");
46         if (action == null) action = "login";
47
48         switch (action) {
49             case "register": // Shows registration page
50                 forward(request, response, "/jsp/register.jsp");
51                 break;
52
53             case "login": // Show login page
54                 forward(request, response, "/jsp/login.jsp");
55                 break;
56
57             case "logout": // Perform logout
58                 doLogout(request, response);
59                 break;
60
61             case "profile": // Show user profile
62                 forward(request, response, "/jsp/customer/profile.jsp");
63                 break;
64
65             default: // Unknown action → go to login page
66                 response.sendRedirect(request.getContextPath() + "/user?action=login");
67         }
68     }
69
70     // -----
71
72     // POST Requests
73     // -----
74     protected void doPost(HttpServletRequest request, HttpServletResponse response)
75         throws ServletException, IOException {
76
77         // Read which action was submitted from the form
78         String action = request.getParameter("action");
79
80         // If no action provided → bad request
81         if (action == null) {
82             response.sendError(HttpServletResponse.SC_BAD_REQUEST);
83             return;
84         }
85
86         switch (action) {
87             case "register": // Process registration form
88                 handleRegister(request, response);
89                 break;
90
91             case "login": // Process login form
92                 handleLogin(request, response);
93                 break;
94
95             case "profileUpdate": // Process profile update
96                 handleProfileUpdate(request, response);
97                 break;
98
99             default:
100                 response.sendError(HttpServletResponse.SC_BAD_REQUEST);
101         }
102     }
103
104     // -----
105     // Registration
106     // -----
```

```
106 private void handleRegister(HttpServletRequest req, HttpServletResponse resp)
107     throws ServletException, IOException {
108
109     // Collect form inputs
110     String name = req.getParameter("name");
111     String username = req.getParameter("username");
112     String email = req.getParameter("email");
113     String phone = req.getParameter("phone");
114     String address = req.getParameter("address");
115     String plainPassword = req.getParameter("password");
116
117
118     username = username.trim();
119     email = email != null ? email.trim() : null;
120
121
122     // NEW: role taken from form
123     String role = req.getParameter("role");
124     if (role == null || role.isEmpty()) role = "customer";
125
126     // Validate required fields
127     if (username == null || username.isEmpty() || plainPassword == null ||
plainPassword.isEmpty()) {
128         req.setAttribute("error", "Username and password are required.");
129         forward(req, resp, "/jsp/register.jsp");
130         return;
131     }
132
133
134
135     // Check if username exists
136     if (userDao.getUserByUsername(username) != null) {
137         req.setAttribute("error", "Registration failed. Please try a different username.");
138         forward(req, resp, "/jsp/register.jsp");
139         return;
140     }
141
142     // Hash password before saving (BCrypt is secure)
143     String hashedPassword = BCrypt.hashpw(plainPassword, BCrypt.gensalt(12));
144
145     // Create user model and populate fields
146     User user = new User();
147     user.setName(name);
148     user.setUsername(username);
149     user.setPassword(hashedPassword);
150     user.setEmail(email);
151     user.setPhone(phone);
152     user.setAddress(address);
153     user.setRole(role);
154
155     // Insert new user into database
156     userDao.addUser(user);
157
158     // Show success message and redirect to login page
159     req.setAttribute("message", "Registration successful. Please login.");
160     forward(req, resp, "/jsp/login.jsp");
161 }
162
163 // -----
164 // Login + RBAC
165 // -----
166 private void handleLogin(HttpServletRequest req, HttpServletResponse resp)
167     throws ServletException, IOException {
168
169     String username = req.getParameter("username");
170     String passwordAttempt = req.getParameter("password");
171
172     // Basic validation
173     if (username == null || passwordAttempt == null) {
174         req.setAttribute("error", "Username and password required.");
175         forward(req, resp, "/jsp/login.jsp");
```

```

176         return;
177     }
178
179     // Retrieve user from DB
180     User user = userDao.getUserByUsername(username);
181
182     // Check if user exists AND password matches
183     if (user == null || !BCrypt.checkpw(passwordAttempt, user.getPassword())) {
184         req.setAttribute("error", "Invalid username or password.");
185         forward(req, resp, "/jsp/login.jsp");
186         return;
187     }
188
189     // Create session and store logged-in user
190     HttpSession session = req.getSession();
191     session.setAttribute("loggedUser", user);
192     session.setMaxInactiveInterval(30 * 60);
193
194     // NEW – store role in session
195     session.setAttribute("role", user.getRole());
196
197     // NEW – redirect based on role
198     String role = user.getRole();
199
200     if ("admin".equalsIgnoreCase(role)) {
201         resp.sendRedirect(req.getContextPath() + "/admin/dashboard");
202     } else if ("delivery".equalsIgnoreCase(role)) {
203         resp.sendRedirect(req.getContextPath() + "/jsp/delivery/dashboard.jsp");
204     } else {
205         resp.sendRedirect(req.getContextPath() + "/jsp/customer/home.jsp");
206     }
207 }
208
209 // -----
210 // Logout
211
212 // -----
213 private void doLogout(HttpServletRequest req, HttpServletResponse resp)
214     throws IOException {
215     // Get current session (don't create new one)
216     HttpSession session = req.getSession(false);
217
218     // Remove user session if exists
219     if (session != null) session.invalidate();
220     resp.sendRedirect(req.getContextPath() + "/user?action=login");
221 }
222
223 // -----
224 // Profile
225 // -----
226 private void showProfile(HttpServletRequest req, HttpServletResponse resp)
227     throws ServletException, IOException {
228     HttpSession session = req.getSession(false);
229
230     // If session invalid or user not logged in → redirect to login
231     if (session == null || session.getAttribute("loggedUser") == null) {
232         resp.sendRedirect(req.getContextPath() + "/user?action=login");
233         return;
234     }
235 }
236
237 // Forward user to profile JSP page
238 forward(req, resp, "/jsp/profile.jsp");
239 }
240
241 // -----
242 // Profile Update
243 // -----
244 private void handleProfileUpdate(HttpServletRequest req, HttpServletResponse resp)
245     throws ServletException, IOException {

```

```

246
247     HttpSession session = req.getSession(false);
248
249     // Ensure user is logged in
250     if (session == null || session.getAttribute("loggedUser") == null) {
251         resp.sendRedirect(req.getContextPath() + "/user?action=login");
252         return;
253     }
254
255     // Get current logged-in user object
256     User logged = (User) session.getAttribute("loggedUser");
257
258     // Update profile fields
259     logged.setName(req.getParameter("name"));
260     logged.setEmail(req.getParameter("email"));
261     logged.setPhone(req.getParameter("phone"));
262     logged.setAddress(req.getParameter("address"));
263
264     // Handle password update only if provided
265     String newPassword = req.getParameter("newPassword");
266     if (newPassword != null && !newPassword.isEmpty()) {
267         logged.setPassword(BCrypt.hashpw(newPassword, BCrypt.gensalt(12)));
268     }
269
270     // Save updated user to DB
271     userDao.updateUser(logged);
272
273     // Update session with new data
274     session.setAttribute("loggedUser", logged);
275
276     // Show success message
277     req.setAttribute("message", "Profile updated successfully.");
278
279     // Stay on profile page
280     forward(req, resp, "/jsp/profile.jsp");
281 }
282
283
284 // -----
285 // Role Checker (Optional but useful)
286 // -----
287 @SuppressWarnings("unused")
288 private boolean hasRole(HttpServletRequest req, String requiredRole) {
289     HttpSession session = req.getSession(false);
290     if (session == null) return false;
291
292     String role = (String) session.getAttribute("role");
293     if (role == null) return false;
294
295     return role.equalsIgnoreCase(requiredRole);
296 }
297
298 // -----
299 // Utility Forward Request to JSP
300 // -----
301 private void forward(HttpServletRequest req, HttpServletResponse resp, String path)
302     throws ServletException, IOException {
303
304     // Use RequestDispatcher to forward request internally (URL doesn't change)
305     RequestDispatcher rd = req.getRequestDispatcher(path);
306     rd.forward(req, resp);
307 }
308 }
309
310
311
312 /*How requests flow
313 ■ GET Requests (doGet)
314
315 Used for displaying pages:

```

```

316
317 Action Page/Logic
318 register /jsp/register.jsp
319 login /jsp/login.jsp
320 logout Calls doLogout()
321 profile Requires user login → forwards to /jsp/profile.jsp
322
323
324 ■ POST Requests (doPost)
325
326 Used when forms submit data:
327
328 Action Method Purpose
329 register handleRegister() Creates new account
330 login handleLogin() Authenticates user
331 profileUpdate handleProfileUpdate() Updates user info
332
333
334 BCrypt Password Hashing: Passwords are always stored in hashed form (secure).
335 hashpw() → convert plain text to hash
336 checkpw() → verify login password
337 This prevents storing real passwords in the database.
338
339
340 Sessions
341 When the user logs in:
342         session.setAttribute("loggedUser", user);
343 This keeps the user logged in until:
344 *They logout
345 *Session expires (30 min)
346 *
347 *
348 *Forwarding vs Redirect
349 * forward() → stays inside servlet, URL unchanged
350 sendRedirect() → instructs browser to load a different URL

```

```

351 *
352 * */
353

```

2. RestaurantServlet.java (URL: /restaurant)

Role / Access Level: Customer / Public

Primary Purpose: Restaurant Listing, Search, and Detailed Profile Discovery Controller.

Architectural Overview: RestaurantServlet provides restaurant discovery features for customers. It retrieves all active Bangladeshi restaurants (e.g., Kacchi Bhai, Sultan's Dine, Star Kabab, Nanna Biryani, Takeout Bangladesh) and fetches individual restaurant menus and customer reviews.

HTTP Methods & Flow Breakdown:

- **doGet():** Handles `action=list` to fetch all restaurants or `action=details` to display a specific restaurant along with its dishes and customer ratings.
- **doPost():** Delegates search queries and restaurant filter actions to `doGet()`.

Key Implementation Highlights:

- **Data Aggregation:** Combines `RestaurantDAO` (restaurant profiles), `MenuDAO` (associated menu dishes), and `ReviewDAO` (customer star ratings and reviews).
- **Request Attributes:** Prepares `\${restaurants}`, `\${restaurant}`, `\${menuitems}`, and `\${reviews}` for consumption by JSP rendering templates (`restaurantList.jsp`, `restaurantDetails.jsp`).

Source Code (RestaurantServlet.java):

```

1 package com.app.controllers;
2
3 import java.io.IOException;
4 import java.util.List;
5
6 import javax.servlet.RequestDispatcher;
7 import javax.servlet.ServletException;
8 import javax.servlet.annotation.WebServlet;
9 import javax.servlet.http.HttpServlet;
10 import javax.servlet.http.HttpServletRequest;
11 import javax.servlet.http.HttpServletResponse;
12
13 import com.app.dao.MenuItemDAO;
14 import com.app.dao.RestaurantDAO;
15 import com.app.dao_implementation.MenuItemDAOImpl;
16 import com.app.dao_implementation.RestaurantDAOImpl;
17 import com.app.models.MenuItem;
18 import com.app.models.Restaurant;
19
20 /*
21  * This RestaurantServlet controls:
22  *
23  * ✓ Listing all restaurants
24  * ✓ Viewing a single restaurant's details
25  * ✓ Redirecting/Forwarding to restaurant JSP pages
26  *
27  * Pattern used:
28  * - doGet → page display (list, view)
29  * - doPost → not used (no form submission yet)
30  */
31
32 @WebServlet("/restaurant")
33 public class RestaurantServlet extends HttpServlet {
34     private static final long serialVersionUID = 1L;
35
36     // DAO object for database operations
37     private RestaurantDAO restaurantDAO;
38
39     @Override
40     public void init() throws ServletException {
41         // Initialize DAO once when servlet starts
42         restaurantDAO = new RestaurantDAOImpl();
43     }
44
45     // -----
46     // GET Requests (display pages)
47     // -----
48     @Override
49     protected void doGet(HttpServletRequest request, HttpServletResponse response)
50         throws ServletException, IOException {
51
52         // Determine which action user wants (default = list restaurants)
53         String action = request.getParameter("action");
54         if (action == null) action = "list";
55
56         switch (action) {
57
58             case "view": // Show a single restaurant details
59                 showRestaurantDetails(request, response);
60                 break;
61
62             case "list": // Show all restaurants
63             default:
64                 listRestaurants(request, response);
65                 break;
66         }
67     }
68
69     // -----
70     // POST Requests (currently unused for restaurants)

```

```

71 // -----
72 @Override
73 protected void doPost(HttpServletRequest request, HttpServletResponse response)
74     throws ServletException, IOException {
75
76     // No POST-use cases yet → return error
77     response.sendError(HttpServletResponse.SC_BAD_REQUEST);
78 }
79
80 // -----
81 // List All Restaurants
82 // URL → /restaurant?action=list
83 // -----
84 private void listRestaurants(HttpServletRequest req, HttpServletResponse resp)
85     throws ServletException, IOException {
86
87     // Get list from database
88     List<Restaurant> restaurants = restaurantDAO.getAllRestaurants();
89
90     // Send list to JSP
91     req.setAttribute("restaurants", restaurants);
92
93     // Forward to the JSP page
94     forward(req, resp, "/jsp/customer/restaurantList.jsp");
95
96 }
97
98 // -----
99 // View Restaurant Details
100 // URL → /restaurant?action=view&id=10
101 // -----
102 private void showRestaurantDetails(HttpServletRequest req, HttpServletResponse resp)
103     throws ServletException, IOException {
104
105     String idParam = req.getParameter("id");
106
107     if (idParam == null) {
108         resp.sendRedirect(req.getContextPath() + "/restaurant?action=list");
109         return;
110     }
111
112     int restaurantId = Integer.parseInt(idParam);
113
114     // 1■■ Restaurant
115     Restaurant restaurant = restaurantDAO.getRestaurantById(restaurantId);
116     if (restaurant == null || !restaurant.isActive()) {
117         resp.sendRedirect(req.getContextPath() + "/restaurant?action=list");
118         return;
119     }
120
121     // 2■■ Menu
122     MenuItemDAO menuItemDAO = new MenuItemDAOImpl();
123     List<MenuItem> menuItems = menuItemDAO.getMenuItemsByRestaurantId(restaurantId);
124
125     // 3■■ Reviews
126     com.app.dao.ReviewDAO reviewDAO = new com.app.dao_implementation.ReviewDAOImpl();
127     List<com.app.models.Review> reviews = reviewDAO.getReviewsByRestaurantId(restaurantId);
128
129     java.util.Map<Integer, com.app.models.User> reviewerMap = new java.util.HashMap<>();
130     com.app.dao.UserDAO userDAO = new com.app.dao_implementation.UserDAOImpl();
131     for (com.app.models.Review r : reviews) {
132         if (!reviewerMap.containsKey(r.getUserId())) {
133             reviewerMap.put(r.getUserId(), userDAO.getUser(r.getUserId()));
134         }
135     }
136
137     // 4■■ Attach
138     req.setAttribute("restaurant", restaurant);
139     req.setAttribute("menuItems", menuItems);
140     req.setAttribute("reviews", reviews);

```

```

141         req.setAttribute("reviewerMap", reviewerMap);
142
143         forward(req, resp, "/jsp/customer/restaurantDetails.jsp");
144     }
145
146     // -----
147     // Utility Function → forward to JSP
148     // -----
149     private void forward(HttpServletRequest req, HttpServletResponse resp, String path)
150         throws ServletException, IOException {
151         RequestDispatcher rd = req.getRequestDispatcher(path);
152         rd.forward(req, resp);
153     }
154 }
155
156
157 /*Action URL Purpose
158 list (default): /restaurant?action=list To Show all restaurants
159 view : /restaurant?action=view&id=3 To Show restaurant details
160 *
161 * **/
162

```

3. CartServlet.java (URL: /cart)

Role / Access Level: Customer

Primary Purpose: Shopping Cart Session State & Price Calculation Engine.

Architectural Overview: CartServlet manages the customer's in-memory shopping cart stored within their active `HttpSession`. It allows customers to add dishes, adjust quantities, remove items, and view price calculations in Bangladeshi Taka (₳).

HTTP Methods & Flow Breakdown:

- **doGet():** Displays current cart contents (`cart.jsp`) with calculated subtotal, delivery fee (₳20), platform fee (₳5), and tax (3%).
- **doPost():** Handles `action=add`, `action=update`, `action=delete`, and `action=clear`. Mutates the `Cart` object stored in the HTTP Session.

Key Implementation Highlights:

- **In-Memory Cart Management:** Instantiates or fetches a `Cart` object from `session.getAttribute("cart")`.
- **Currency Consistency:** Computes item subtotals and grand total formatted in Taka (₳).
- **Synchronization:** Ensures cart updates dynamically persist across page navigations before final checkout.

Source Code (CartServlet.java):

```
1 package com.app.controllers;
2
3 import java.io.IOException;
4 import java.util.List;
5
6 import javax.servlet.*;
7 import javax.servlet.annotation.WebServlet;
8 import javax.servlet.http.*;
9
10 import com.app.dao.CartDAO;
11 import com.app.dao.MenuItemDAO;
12 import com.app.dao_implementation.CartDAOImpl;
13 import com.app.dao_implementation.MenuItemDAOImpl;
14 import com.app.models.Cart;
15 import com.app.models.CartItem;
16 import com.app.models.MenuItem;
17 import com.app.models.User;
18
19 @WebServlet("/cart")
20 public class CartServlet extends HttpServlet {
21
22     private CartDAO cartDAO = new CartDAOImpl();
23     private MenuItemDAO menuItemDAO = new MenuItemDAOImpl();
24
25     @Override
26     protected void doGet(HttpServletRequest req, HttpServletResponse resp)
27         throws ServletException, IOException {
28
29         String action = req.getParameter("action");
30
31         if (action == null) action = "view";
32
33         switch (action) {
34             case "view":
35                 showCart(req, resp);
36
37                 break;
38             case "remove":
39                 removeItem(req, resp);
40                 break;
41             case "clear":
42                 clearCart(req, resp);
43                 break;
44             default:
45                 showCart(req, resp);
46         }
47     }
48
49     @Override
50     protected void doPost(HttpServletRequest req, HttpServletResponse resp)
51         throws ServletException, IOException {
52
53         String action = req.getParameter("action");
54
55         if (action == null) action = "view";
56
57         switch (action) {
58             case "add":
59                 addItem(req, resp);
60                 break;
61             case "update":
62                 updateItem(req, resp);
63                 break;
64             default:
65                 resp.sendRedirect("cart?action=view");
66         }
67     }
68
69 }
```

```

71     }
72
73     // =====
74     // VIEW CART / show Cart
75     // =====
76     private void showCart(HttpServletRequest req, HttpServletResponse resp)
77         throws ServletException, IOException {
78
79         HttpSession session = req.getSession(false);
80         User loggedUser = session != null ? (User) session.getAttribute("loggedUser") : null;
81         if (loggedUser == null) {
82             resp.sendRedirect(req.getContextPath() + "/user?action=login");
83             return;
84         }
85         int userId = loggedUser.getUserId();
86
87         // Ensure cart exists
88         Cart cart = cartDAO.getCartByUserId(userId);
89         if (cart == null) {
90             cartDAO.createCart(userId);
91             cart = cartDAO.getCartByUserId(userId);
92         }
93
94         List<CartItem> cartItems = cartDAO.getCartItems(cart.getCartId());
95
96         // Fetch MenuItem details for display (name, price, image)
97         java.util.Map<Integer, MenuItem> menuMap = new java.util.HashMap<>();
98
99         for (CartItem ci : cartItems) {
100             MenuItem item = menuItemDAO.getMenuItemById(ci.getMenuItemId());
101             if (item != null) menuMap.put(ci.getMenuItemId(), item);
102         }
103
104         double total = cartDAO.getCartTotal(cart.getCartId());
105
106         req.setAttribute("cartItems", cartItems);
107         req.setAttribute("menuItems", menuMap);
108         req.setAttribute("cartTotal", total);
109
110         RequestDispatcher rd =
111             req.getRequestDispatcher("/jsp/customer/cart.jsp");
112         rd.forward(req, resp);
113     }
114 }
115
116 // =====
117 // ADD ITEM TO CART
118 // =====
119 private void addItem(HttpServletRequest req, HttpServletResponse resp)
120     throws IOException {
121
122     HttpSession session = req.getSession(false);
123     User loggedUser = session != null ? (User) session.getAttribute("loggedUser") : null;
124     if (loggedUser == null) {
125         // If AJAX, return 401 JSON. If normal, redirect.
126         if (isAjax(req)) {
127             resp.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
128             resp.setContentType("application/json");
129             resp.getWriter().write("{\"success\":false, \"error\":\"not_authenticated\"}");
130             return;
131         } else {
132             resp.sendRedirect(req.getContextPath() + "/user?action=login");
133             return;
134         }
135     }
136     int userId = loggedUser.getUserId();
137
138     int menuItemId;
139     int quantity;
140     try {

```

```

141         menuItemId = Integer.parseInt(req.getParameter("menuItemId"));
142         quantity = Integer.parseInt(req.getParameter("quantity"));
143     } catch (NumberFormatException e) {
144         if (isAjax(req)) {
145             resp.setStatus(HttpServletResponse.SC_BAD_REQUEST);
146             resp.setContentType("application/json");
147             resp.getWriter().write("{\"success\":false, \"error\":\"invalid_parameters\"}");
148             return;
149         } else {
150             resp.sendRedirect("cart?action=view");
151             return;
152         }
153     }
154
155     // Ensure cart exists
156     Cart cart = cartDAO.getCartByUserId(userId);
157     if (cart == null) cartDAO.createCart(userId);
158
159     // Perform DB insert/update
160     cartDAO.addItemToCart(userId, menuItemId, quantity);
161
162     // If AJAX: return JSON success and total item count
163     if (isAjax(req)) {
164         cart = cartDAO.getCartByUserId(userId);
165         int cartCount = 0;
166         if (cart != null) {
167             for (CartItem ci : cartDAO.getCartItems(cart.getCartId())) {
168                 cartCount += ci.getQuantity();
169             }
170         }
171         resp.setContentType("application/json");
172         resp.setCharacterEncoding("UTF-8");
173         resp.getWriter().write("{\"success\":true, \"cartCount\":" + cartCount + "}");
174         return;
175     }
176
177     // Non-AJAX: redirect to view cart
178     resp.sendRedirect("cart?action=view");
179 }
180
181 // =====
182 // UPDATE ITEM QUANTITY
183 // =====
184 private void updateItem(HttpServletRequest req, HttpServletResponse resp)
185     throws IOException {
186
187     HttpSession session = req.getSession(false);
188     User loggedUser = session != null ? (User) session.getAttribute("loggedUser") : null;
189     if (loggedUser == null) {
190         resp.sendRedirect(req.getContextPath() + "/user?action=login");
191         return;
192     }
193     int userId = loggedUser.getUserId();
194
195     int menuItemId = Integer.parseInt(req.getParameter("menuItemId"));
196     int quantity = Integer.parseInt(req.getParameter("quantity"));
197
198     Cart cart = cartDAO.getCartByUserId(userId);
199
200     if (cart != null) {
201         cartDAO.updateCartItemQuantity(cart.getCartId(), menuItemId, quantity);
202     }
203
204     resp.sendRedirect("cart?action=view");
205 }
206
207 // =====
208 // REMOVE ITEM FROM CART
209 // =====
210 private void removeItem(HttpServletRequest req, HttpServletResponse resp)

```

```

211         throws IOException {
212
213         HttpSession session = req.getSession(false);
214         User loggedUser = session != null ? (User) session.getAttribute("loggedUser") : null;
215         if (loggedUser == null) {
216             resp.sendRedirect(req.getContextPath() + "/user?action=login");
217             return;
218         }
219         int userId = loggedUser.getUserId();
220
221         int menuItemId = Integer.parseInt(req.getParameter("menuItemId"));
222
223         Cart cart = cartDAO.getCartByUserId(userId);
224
225         if (cart != null) {
226             cartDAO.removeItemFromCart(cart.getCartId(), menuItemId);
227         }
228
229         resp.sendRedirect("cart?action=view");
230     }
231
232     // =====
233     // CLEAR CART
234     // =====
235     private void clearCart(HttpServletRequest req, HttpServletResponse resp)
236         throws IOException {
237
238         HttpSession session = req.getSession(false);
239         User loggedUser = session != null ? (User) session.getAttribute("loggedUser") : null;
240         if (loggedUser == null) {
241             resp.sendRedirect(req.getContextPath() + "/user?action=login");
242             return;
243         }
244         int userId = loggedUser.getUserId();
245
246         Cart cart = cartDAO.getCartByUserId(userId);
247
248         if (cart != null) {
249             cartDAO.clearCart(cart.getCartId());
250         }
251
252         resp.sendRedirect("cart?action=view");
253     }
254
255     // =====
256     // Utility: detect AJAX requests
257     // =====
258     private boolean isAjax(HttpServletRequest req) {
259         String xhr = req.getHeader("X-Requested-With");
260         return xhr != null && "XMLHttpRequest".equalsIgnoreCase(xhr);
261     }
262 }
263

```

4. CheckoutServlet.java (URL: /checkout)

Role / Access Level: Customer

Primary Purpose: Order Placement, Address Assignment, and Transaction Persistence Controller.

Architectural Overview: CheckoutServlet transitions active shopping cart items into an official persistent Order record in MySQL. It validates user authentication, verifies cart contents, assigns delivery addresses, creates order items, and flushes the active cart.

HTTP Methods & Flow Breakdown:

- **doGet():** Renders order confirmation and address selection screen (`checkout.jsp`).
- **doPost():** Validates session, creates `Order` entity with status `PENDING`, saves order via `OrderDAO.createOrder()`, persists item details via `OrderItemDAO`, clears cart, and redirects to

`/orderSuccess`.

Key Implementation Highlights:

- Transactional Flow: Inserts parent record in `orders` table and child records in `order\_items` table.
- Address & Payment Binding: Associates selected delivery address ID and chosen payment method (bKash/Nagad/COD/Card).
- Session Clearance: Resets active session cart upon successful order creation to prevent duplicate submission.

Source Code (CheckoutServlet.java):

```
1 package com.app.controllers;
2
3 import java.io.IOException;
4 import java.util.*;
5
6 import javax.servlet.*;
7 import javax.servlet.annotation.WebServlet;
8 import javax.servlet.http.*;
9
10 import com.app.dao.CartDAO;
11 import com.app.dao.MenuItemDAO;
12 import com.app.dao.OrderDAO;
13 import com.app.dao_implementation.CartDAOImpl;
14 import com.app.dao_implementation.MenuItemDAOImpl;
15 import com.app.dao_implementation.OrderDAOImpl;
16
17 import com.app.models.Cart;
18 import com.app.models.CartItem;
19 import com.app.models.MenuItem;
20 import com.app.models.Order;
21 import com.app.models.OrderItem;
22 import com.app.models.User;
23
24 @WebServlet("/checkout")
25 public class CheckoutServlet extends HttpServlet {
26
27     private final CartDAO cartDAO = new CartDAOImpl();
28     private final MenuItemDAO menuItemDAO = new MenuItemDAOImpl();
29     private final OrderDAO orderDAO = new OrderDAOImpl();
30
31     /* =====
32     GET : SHOW CHECKOUT PAGE
33     ===== */
34     @Override
35     protected void doGet(HttpServletRequest req, HttpServletResponse resp)
```

```

36         throws ServletException, IOException {
37
38         HttpSession session = req.getSession(false);
39
40         // ■ SINGLE SOURCE OF AUTH TRUTH
41         User user = (session == null)
42             ? null
43             : (User) session.getAttribute("loggedUser");
44
45         // ■ NOT LOGGED IN
46         if (user == null) {
47             resp.sendRedirect(req.getContextPath() + "/user?action=login");
48             return;
49         }
50
51         int userId = user.getUserId();
52
53         Cart cart = cartDAO.getCartByUserId(userId);
54
55         // ■ CART NOT FOUND / EMPTY
56         if (cart == null) {
57             resp.sendRedirect(req.getContextPath() + "/cart?action=view");
58             return;
59         }
60
61         List<CartItem> cartItems = cartDAO.getCartItems(cart.getCartId());
62
63         if (cartItems == null || cartItems.isEmpty()) {
64             resp.sendRedirect(req.getContextPath() + "/cart?action=view");
65             return;
66         }
67
68         double totalAmount = cartDAO.getCartTotal(cart.getCartId());
69
70         // ■ BUILD MENU ITEM MAP
71
72         Map<Integer, MenuItem> menuItems = new HashMap<>();
73         for (CartItem ci : cartItems) {
74             menuItems.put(ci.getMenuItemId(),
75                 menuItemDAO.getMenuItemById(ci.getMenuItemId()));
76         }
77
78         // ■ SET ATTRIBUTES
79         req.setAttribute("cartItems", cartItems);
80         req.setAttribute("menuItems", menuItems);
81         req.setAttribute("totalAmount", totalAmount);
82
83         // ■ FORWARD TO JSP (NO REDIRECT)
84         req.getRequestDispatcher("/jsp/customer/checkout.jsp")
85             .forward(req, resp);
86     }
87
88     /* =====
89     POST : PLACE ORDER
90     ===== */
91     @Override
92     protected void doPost(HttpServletRequest req, HttpServletResponse resp)
93         throws ServletException, IOException {
94
95         HttpSession session = req.getSession(false);
96
97         // ■ SINGLE SOURCE OF AUTH TRUTH
98         User user = (session == null)
99             ? null
100             : (User) session.getAttribute("loggedUser");
101
102         if (user == null) {
103             resp.sendRedirect(req.getContextPath() + "/user?action=login");
104             return;
105         }

```

```

106         int userId = user.getUserId();
107
108         Cart cart = cartDAO.getCartByUserId(userId);
109
110         if (cart == null) {
111             resp.sendRedirect(req.getContextPath() + "/cart?action=view");
112             return;
113         }
114
115         List<CartItem> cartItems = cartDAO.getCartItems(cart.getCartId());
116
117         if (cartItems == null || cartItems.isEmpty()) {
118             resp.sendRedirect(req.getContextPath() + "/cart?action=view");
119             return;
120         }
121
122         /* ===== READ CHECKOUT FORM ===== */
123         /* ===== READ CHECKOUT FORM ===== */
124         int restaurantId = Integer.parseInt(req.getParameter("restaurantId"));
125
126         String paymentMethod = req.getParameter("payment_method");
127         if (paymentMethod == null || paymentMethod.isEmpty()) {
128             paymentMethod = "COD"; // fallback safety
129         }
130
131         String deliveryInstructions = req.getParameter("instructions");
132
133         /* ■ SAFE ADDRESS PARSING */
134         String addressIdStr = req.getParameter("addressId");
135         Integer addressId = null;
136         if (addressIdStr != null && !addressIdStr.trim().isEmpty()) {
137             addressId = Integer.parseInt(addressIdStr);
138         }
139
140         double totalAmount = cartDAO.getCartTotal(cart.getCartId());
141
142
143         /* ===== CREATE ORDER ===== */
144         Order order = new Order();
145         order.setUserId(userId);
146         order.setRestaurantId(restaurantId);
147         order.setAddressId(addressId);
148         order.setTotalAmount(totalAmount);
149         order.setStatus("placed");
150         order.setPaymentMethod(paymentMethod);
151         order.setPaymentStatus("pending");
152         order.setDeliveryInstructions(deliveryInstructions);
153
154         /* ===== ORDER ITEMS ===== */
155         List<OrderItem> orderItems = new ArrayList<>();
156         for (CartItem ci : cartItems) {
157             MenuItem mi = menuItemDAO.getMenuItemById(ci.getMenuItemId());
158
159             OrderItem oi = new OrderItem();
160             oi.setMenuItemId(mi.getMenuItemId());
161             oi.setQuantity(ci.getQuantity());
162             oi.setPriceAtOrder(mi.getPrice());
163
164             orderItems.add(oi);
165         }
166
167         int orderId = orderDAO.placeOrder(order, orderItems);
168
169         if (orderId <= 0) {
170             req.setAttribute("errorMessage",
171                 "Order could not be placed due to a system error.");
172             req.getRequestDispatcher("/jsp/customer/checkout.jsp")
173                 .forward(req, resp);
174             return;
175         }

```

```

176
177      /* ===== CLEANUP & REDIRECT ===== */
178      cartDAO.clearCart(cart.getCartId());
179
180      resp.sendRedirect(req.getContextPath()
181          + "/orderSuccess?orderId=" + orderId);
182  }
183 }
184

```

5. OrderHistoryServlet.java (URL: /orderHistory)

Role / Access Level: Customer

Primary Purpose: Customer Order History Timeline & Tracking Controller.

Architectural Overview: OrderHistoryServlet retrieves and displays past orders placed by the currently logged-in customer. It presents order statuses (`PENDING`, `PREPARING`, `OUT\_FOR\_DELIVERY`, `DELIVERED`, `CANCELLED`) with itemized details.

HTTP Methods & Flow Breakdown:

- **doGet():** Extracts `userId` from session, queries `OrderDAO.getOrdersByUserId(userId)`, attaches order list to request scope, and forwards to `orderHistory.jsp`.
- **doPost():** Delegates processing to `doGet()`.

Key Implementation Highlights:

- **Security Isolation:** Restricts query results strictly to the logged-in customer's `userId`.
- **Status Mapping:** Provides real-time order tracking statuses updated by system administrators.

Source Code (OrderHistoryServlet.java):

```

1 package com.app.controllers;
2
3 import com.app.dao.OrderDAO;
4 import com.app.dao_implementation.OrderDAOImpl;
5 import com.app.dao.MenuItemDAO;
6 import com.app.dao_implementation.MenuItemDAOImpl;
7 import com.app.dao.RestaurantDAO;
8 import com.app.dao_implementation.RestaurantDAOImpl;
9 import com.app.models.*;
10
11 import javax.servlet.ServletException;
12 import javax.servlet.annotation.WebServlet;
13 import javax.servlet.http.*;
14 import java.io.IOException;
15 import java.util.*;
16
17 @WebServlet("/orderHistory")
18 public class OrderHistoryServlet extends HttpServlet {
19
20     private final OrderDAO orderDAO = new OrderDAOImpl();
21     private final MenuItemDAO menuItemDAO = new MenuItemDAOImpl();
22     private final RestaurantDAO restaurantDAO = new RestaurantDAOImpl(); // New DAO
23
24     @Override
25     protected void doGet(HttpServletRequest req, HttpServletResponse resp)
26         throws ServletException, IOException {
27
28         HttpSession session = req.getSession(false);
29         User loggedUser = (session != null) ? (User) session.getAttribute("loggedUser") : null;
30
31         if (loggedUser == null) {
32             resp.sendRedirect(req.getContextPath() + "/user?action=login");
33             return;
34         }
35

```

```

36     int userId = loggedUser.getUserId();
37
38     // 1■■■ Fetch all orders for this user
39     List<Order> orders = orderDAO.getOrdersByUserId(userId);
40     req.setAttribute("orders", orders);
41
42     // 2■■■ Prepare orderItems mapping and collect menuIds
43     Map<Integer, List<OrderItem>> orderItemsMap = new HashMap<>();
44     Set<Integer> menuIds = new HashSet<>();
45     Set<Integer> restaurantIds = new HashSet<>();
46
47     for (Order o : orders) {
48         // Collect restaurant IDs for map
49         restaurantIds.add(o.getRestaurantId());
50
51         // Fetch order items
52         List<OrderItem> items = orderDAO.getOrderItems(o.getOrderId());
53         orderItemsMap.put(o.getOrderId(), items);
54
55         for (OrderItem it : items) {
56             menuIds.add(it.getMenuItemId());
57         }
58     }
59     req.setAttribute("orderItemsMap", orderItemsMap);
60
61     // 3■■■ Prepare MenuItem map for JSP (name, price, image)
62     Map<Integer, MenuItem> menuItemsMap = menuItemDAO.getMenuItemsAsMap(menuIds);
63     req.setAttribute("menuItemsMap", menuItemsMap);
64
65     // 4■■■ Prepare Restaurant map for JSP (restaurantId → restaurantName)
66     Map<Integer, String> restaurantsMap = new HashMap<>();
67     for (Integer rid : restaurantIds) {
68         Restaurant r = restaurantDAO.getRestaurantById(rid);
69         if (r != null) {
70             restaurantsMap.put(rid, r.getName());
71         }
72     }
73     req.setAttribute("restaurantsMap", restaurantsMap);
74
75     // 5■■■ Forward to JSP
76     req.getRequestDispatcher("/jsp/customer/orderHistory.jsp").forward(req, resp);
77 }
78 }
79

```

6. ReviewServlet.java (URL: /review)

Role / Access Level: Customer

Primary Purpose: Customer Rating (1-5 Stars) & Feedback Submission Controller.

Architectural Overview: ReviewServlet handles customer rating and review comments for Bangladeshi restaurants. Upon review submission, it triggers an automated recalculation of the restaurant's overall average star rating in the database.

HTTP Methods & Flow Breakdown:

- **doGet():** Redirects to restaurant details view.
- **doPost():** Validates logged-in customer session, reads `restaurantId`, `rating` (1-5), and `comment`. Persists review via `ReviewDAO.addReview()` and updates restaurant average rating.

Key Implementation Highlights:

- **Automated Recalculation:** Calls `ReviewDAO.updateRestaurantAverageRating(restaurantId)` to immediately compute and store updated average star ratings.
- **Input Validation:** Ensures ratings are bounded between 1 and 5 stars and comments are properly sanitized.

Source Code (ReviewServlet.java):

```

1 package com.app.controllers;
2
3 import com.app.dao.ReviewDAO;
4 import com.app.dao_implementation.ReviewDAOImpl;
5 import com.app.models.Review;
6 import com.app.models.User;
7
8 import javax.servlet.ServletException;
9 import javax.servlet.annotation.WebServlet;
10 import javax.servlet.http.*;
11 import java.io.IOException;
12
13 @WebServlet("/review")
14 public class ReviewServlet extends HttpServlet {
15
16     private static final long serialVersionUID = 1L;
17     private ReviewDAO reviewDAO;
18
19     @Override
20     public void init() throws ServletException {
21         reviewDAO = new ReviewDAOImpl();
22     }
23
24     @Override
25     protected void doPost(HttpServletRequest req, HttpServletResponse resp)
26         throws ServletException, IOException {
27
28         HttpSession session = req.getSession(false);
29         User user = (session != null) ? (User) session.getAttribute("loggedUser") : null;
30
31         if (user == null) {
32             resp.sendRedirect(req.getContextPath() + "/user?action=login");
33             return;
34         }
35
36         String action = req.getParameter("action");
37         if ("add".equals(action)) {
38             int restaurantId = Integer.parseInt(req.getParameter("restaurantId"));
39             double rating = Double.parseDouble(req.getParameter("rating"));
40             String comment = req.getParameter("comment");
41
42             String orderIdStr = req.getParameter("orderId");
43             Integer orderId = (orderIdStr != null && !orderIdStr.trim().isEmpty())
44                 ? Integer.parseInt(orderIdStr) : null;
45
46             Review review = new Review();
47             review.setUserId(user.getUserId());
48             review.setRestaurantId(restaurantId);
49             review.setOrderId(orderId);
50             review.setRating(rating);
51             review.setComment(comment);
52
53             reviewDAO.addReview(review);
54
55             if (orderId != null) {
56                 resp.sendRedirect(req.getContextPath() + "/orderHistory");
57             } else {
58                 resp.sendRedirect(req.getContextPath() + "/restaurant?action=details&restaurantId=" +
59 restaurantId);
60             }
61         }
62     }
63 }

```

7. AddressServlet.java (URL: /address)

Role / Access Level: Customer

Primary Purpose: Customer Delivery Address CRUD Controller.

Architectural Overview: AddressServlet manages customer delivery address entries (street, city, zip code, landmark). It enables users to save multiple delivery locations for quick selection during checkout.

HTTP Methods & Flow Breakdown:

- **doGet():** Lists existing saved addresses for logged-in user or renders address creation form.
- **doPost():** Handles adding (`action=add`), editing (`action=edit`), or deleting (`action=delete`) address records via `AddressDAO`.

Key Implementation Highlights:

- **User Binding:** Links every saved address directly to the authenticated customer's `userId`.
- **Address Reusability:** Allows saved addresses to be referenced seamlessly during order checkout.

Source Code (AddressServlet.java):

```
1 package com.app.controllers;
2
3 import com.app.dao.AddressDAO;
4 import com.app.dao_implementation.AddressDAOImpl;
5 import com.app.models.Address;
6 import com.app.models.User;
7
8 import javax.servlet.ServletException;
9 import javax.servlet.annotation.WebServlet;
10 import javax.servlet.http.*;
11 import java.io.IOException;
12 import java.util.List;
13
14 /*
15  * AddressServlet
16  * -----
17  * This servlet handles:
18  *
19  * ✓ Viewing all saved addresses of logged-in customer
20  * ✓ Adding a new address
21  * ✓ Editing an existing address
22  * ✓ Deleting an address
23  *
24  * SECURITY:
25  * ✓ Requires user login
26  * ✓ User can access ONLY their own addresses
27  *
28  * URL Mapping:
29  * /customer/addresses
30  */
31
32 @WebServlet("/customer/addresses")
33 public class AddressServlet extends HttpServlet {
34
35     private static final long serialVersionUID = 1L;
```

```

36
37 // DAO for database access
38 private AddressDAO addressDAO;
39
40 @Override
41 public void init() throws ServletException {
42     // Initialize DAO once when servlet loads
43     addressDAO = new AddressDAOImpl();
44 }
45
46 // -----
47 // GET REQUESTS → Show addresses / load edit form
48 // -----
49 @Override
50 protected void doGet(HttpServletRequest req, HttpServletResponse resp)
51     throws ServletException, IOException {
52
53     // 1■■ AUTHENTICATION CHECK
54     HttpSession session = req.getSession(false);
55     User user = (session != null) ? (User) session.getAttribute("loggedUser") : null;
56
57     if (user == null) {
58         // User not logged in → redirect to login
59         resp.sendRedirect(req.getContextPath() + "/user?action=login");
60         return;
61     }
62
63     int userId = user.getUserId(); // ■ Correct and secure
64
65     // 2■■ READ REQUEST PARAMETERS
66     String action = req.getParameter("action");
67     String addressIdParam = req.getParameter("id");
68
69     // 3■■ IF EDIT REQUEST → LOAD ADDRESS
70     if ("edit".equalsIgnoreCase(action) && addressIdParam != null) {
71
72         int addressId = Integer.parseInt(addressIdParam);
73
74         // ■ SECURITY: Fetch address ONLY if it belongs to this user
75         Address editAddress = addressDAO.getAddressById(addressId, userId);
76
77         if (editAddress != null) {
78             req.setAttribute("editAddress", editAddress);
79         }
80     }
81
82     // 4■■ FETCH ALL ADDRESSES OF LOGGED-IN USER
83     List<Address> addresses = addressDAO.getAddressesByUserId(userId);
84     req.setAttribute("addresses", addresses);
85
86     // 5■■ FORWARD TO JSP
87     req.getRequestDispatcher("/jsp/customer/addresses.jsp")
88         .forward(req, resp);
89 }
90
91 // -----
92 // POST REQUESTS → Add / Update / Delete Address
93 // -----
94 @Override
95 protected void doPost(HttpServletRequest req, HttpServletResponse resp)
96     throws IOException {
97
98     // 1■■ AUTHENTICATION CHECK
99     HttpSession session = req.getSession(false);
100     User user = (session != null) ? (User) session.getAttribute("loggedUser") : null;
101
102     if (user == null) {
103         resp.sendRedirect(req.getContextPath() + "/user?action=login");
104         return;
105     }

```

```

106
107     int userId = user.getUserId();
108
109     // 2■■ READ ACTION
110     String action = req.getParameter("action");
111
112     // -----
113     // DELETE ADDRESS
114     // -----
115     if ("delete".equalsIgnoreCase(action)) {
116
117         int addressId = Integer.parseInt(req.getParameter("id"));
118
119         // ■ SECURITY: delete only user's address
120         addressDAO.deleteAddress(addressId, userId);
121
122         resp.sendRedirect(req.getContextPath() + "/customer/addresses");
123         return;
124     }
125
126     // -----
127     // COMMON FIELDS (Add / Update)
128     // -----
129     String street = req.getParameter("street");
130     String city = req.getParameter("city");
131     String state = req.getParameter("state");
132     String zip = req.getParameter("zip");
133     String landmark = req.getParameter("landmark");
134
135     // -----
136     // UPDATE ADDRESS
137     // -----
138     if ("update".equalsIgnoreCase(action)) {
139
140         int addressId = Integer.parseInt(req.getParameter("id"));
141
142         Address address = new Address();
143         address.setAddressId(addressId);
144         address.setUserId(userId);
145         address.setStreet(street);
146         address.setCity(city);
147         address.setState(state);
148         address.setZip(zip);
149         address.setLandmark(landmark);
150
151         addressDAO.updateAddress(address);
152
153         resp.sendRedirect(req.getContextPath() + "/customer/addresses");
154         return;
155     }
156
157     // -----
158     // ADD NEW ADDRESS
159     // -----
160     if ("add".equalsIgnoreCase(action)) {
161
162         Address address = new Address();
163         address.setUserId(userId);
164         address.setStreet(street);
165         address.setCity(city);
166         address.setState(state);
167         address.setZip(zip);
168         address.setLandmark(landmark);
169
170         addressDAO.addAddress(address);
171
172         resp.sendRedirect(req.getContextPath() + "/customer/addresses");
173     }
174 }
175 }

```

```

176

```

8. AdminDashboardServlet.java (URL: /admin/dashboard)

Role / Access Level: Admin Only

Primary Purpose: Administrator Analytics & Platform Summary Controller.

Architectural Overview: AdminDashboardServlet serves as the executive summary dashboard for FoodXpress administrators. It aggregates real-time metrics across total platform revenue (■), total completed orders, active Bangladeshi restaurants, and registered users.

HTTP Methods & Flow Breakdown:

- **doGet():** Executes database aggregations using ``OrderDAO``, ``RestaurantDAO``, and ``UserDAO``. Attaches metrics (``totalRevenue``, ``totalOrders``, ``activeRestaurants``, ``totalUsers``) to request scope and forwards to ``/jsp/admin/dashboard.jsp``.
- **doPost():** Delegates processing to ``doGet()``.

Key Implementation Highlights:

- **Analytics Aggregation:** Calculates live sum of non-cancelled orders in Taka (■).
- **Access Control:** Protected by admin role check (``session.getAttribute("userRole") == "admin"``).

Source Code (AdminDashboardServlet.java):

```
1 package com.app.controllers;
2
3 import com.app.dao.OrderDAO;
4 import com.app.dao.RestaurantDAO;
5 import com.app.dao.UserDAO;
6 import com.app.dao_implementation.OrderDAOImpl;
7 import com.app.dao_implementation.RestaurantDAOImpl;
8 import com.app.dao_implementation.UserDAOImpl;
9 import com.app.models.Order;
10
11 import javax.servlet.ServletException;
12 import javax.servlet.annotation.WebServlet;
13 import javax.servlet.http.*;
14 import java.io.IOException;
15 import java.util.List;
16
17 @WebServlet("/admin/dashboard")
18 public class AdminDashboardServlet extends HttpServlet {
19
20     private static final long serialVersionUID = 1L;
21     private OrderDAO orderDAO;
22     private RestaurantDAO restaurantDAO;
23     private UserDAO userDAO;
24
25     @Override
26     public void init() throws ServletException {
27         orderDAO = new OrderDAOImpl();
28         restaurantDAO = new RestaurantDAOImpl();
29         userDAO = new UserDAOImpl();
30     }
31
32     private boolean isAdmin(HttpServletRequest req) {
33         HttpSession session = req.getSession(false);
34         com.app.models.User u = (session != null) ? (com.app.models.User)
35             session.getAttribute("loggedUser") : null;
36         return u != null && "admin".equalsIgnoreCase(u.getRole());
37     }
38 }
```

```
36     }
37
38     @Override
39     protected void doGet(HttpServletRequest req, HttpServletResponse resp)
40         throws ServletException, IOException {
41
42         if (!isAdmin(req)) {
43             resp.sendRedirect(req.getContextPath() + "/user?action=login");
44             return;
45         }
46
47         List<Order> orders = orderDAO.getAllOrders();
48         double totalRevenue = 0;
49         for (Order o : orders) {
50             // Count all placed/completed orders
51             totalRevenue += o.getTotalAmount();
52         }
53
54         int totalOrders = orders.size();
55         int activeRestaurants = restaurantDAO.getAllRestaurants().size();
56         int totalUsers = userDAO.getAllUsers().size();
57
58         req.setAttribute("totalRevenue", totalRevenue);
59         req.setAttribute("totalOrders", totalOrders);
60         req.setAttribute("activeRestaurants", activeRestaurants);
61         req.setAttribute("totalUsers", totalUsers);
62
63         req.getRequestDispatcher("/jsp/admin/dashboard.jsp").forward(req, resp);
64     }
65 }
66
```

9. AdminOrderServlet.java (URL: /admin/orders)

Role / Access Level: Admin Only

Primary Purpose: Administrator Order Dispatching & Live Status Management Controller.

Architectural Overview: AdminOrderServlet allows administrators to oversee all platform orders across all restaurants and users, allowing live status updates (`PENDING` → `PREPARING` → `OUT\_FOR\_DELIVERY` → `DELIVERED`).

HTTP Methods & Flow Breakdown:

- **doGet():** Fetches all system orders via `OrderDAO.getAllOrders()` and displays them on `orderListAdmin.jsp`.
- **doPost():** Accepts `orderId` and `status`, updates MySQL order status via `OrderDAO.updateOrderStatus()`, and redirects back to order list.

Key Implementation Highlights:

- **Live Status Dispatching:** Enables real-time workflow tracking from kitchen prep to customer delivery.
- **Cross-Platform Visibility:** Provides complete administrative audit trail for all customer orders.

Source Code (AdminOrderServlet.java):

```

1 package com.app.controllers;
2
3 import com.app.dao.OrderDAO;
4 import com.app.dao.RestaurantDAO;
5 import com.app.dao.UserDAO;
6 import com.app.dao_implementation.OrderDAOImpl;
7 import com.app.dao_implementation.RestaurantDAOImpl;
8 import com.app.dao_implementation.UserDAOImpl;
9 import com.app.models.Order;
10 import com.app.models.Restaurant;
11 import com.app.models.User;
12
13 import javax.servlet.ServletException;
14 import javax.servlet.annotation.WebServlet;
15 import javax.servlet.http.*;
16 import java.io.IOException;
17 import java.util.HashMap;
18 import java.util.List;
19 import java.util.Map;
20
21 @WebServlet("/admin/orders")
22 public class AdminOrderServlet extends HttpServlet {
23
24     private static final long serialVersionUID = 1L;
25     private OrderDAO orderDAO;
26     private UserDAO userDAO;
27     private RestaurantDAO restaurantDAO;
28
29     @Override
30     public void init() throws ServletException {
31         orderDAO = new OrderDAOImpl();
32         userDAO = new UserDAOImpl();
33         restaurantDAO = new RestaurantDAOImpl();
34     }
35
36     private boolean isAdmin(HttpServletRequest req) {
37         HttpSession session = req.getSession(false);
38         User u = (session != null) ? (User) session.getAttribute("loggedUser") : null;
39         return u != null && "admin".equalsIgnoreCase(u.getRole());
40     }
41
42     @Override
43     protected void doGet(HttpServletRequest req, HttpServletResponse resp)
44         throws ServletException, IOException {
45
46         if (!isAdmin(req)) {
47             resp.sendRedirect(req.getContextPath() + "/user?action=login");
48             return;
49         }
50
51         String action = req.getParameter("action");
52         if (action == null) action = "list";
53
54         switch (action) {
55             case "updateStatus":
56                 int orderId = Integer.parseInt(req.getParameter("orderId"));
57                 String status = req.getParameter("status");
58                 if (status != null && !status.trim().isEmpty()) {
59                     orderDAO.updateOrderStatus(orderId, status);
60                 }
61                 resp.sendRedirect(req.getContextPath() + "/admin/orders");
62                 break;
63
64             default: // list
65                 List<Order> orderList = orderDAO.getAllOrders();
66                 Map<Integer, User> userMap = new HashMap<>();
67                 Map<Integer, Restaurant> restaurantMap = new HashMap<>();
68
69                 for (Order o : orderList) {
70                     if (!userMap.containsKey(o.getUserId())) {

```

```
71         userMap.put(o.getUserId(), userDAO.getUser(o.getUserId()));
72     }
73     if (!restaurantMap.containsKey(o.getRestaurantId())) {
74         restaurantMap.put(o.getRestaurantId(),
75         restaurantDAO.getRestaurantById(o.getRestaurantId()));
76     }
77
78     req.setAttribute("orders", orderList);
79     req.setAttribute("userMap", userMap);
80     req.setAttribute("restaurantMap", restaurantMap);
81     req.getRequestDispatcher("/jsp/admin/orderListAdmin.jsp").forward(req, resp);
82     break;
83 }
84 }
85
86 @Override
87 protected void doPost(HttpServletRequest req, HttpServletResponse resp)
88     throws ServletException, IOException {
89     doGet(req, resp);
90 }
91 }
92
```

10. AdminUserServlet.java (URL: /admin/users)

Role / Access Level: Admin Only

Primary Purpose: User Account Management & Role Elevation Controller.

Architectural Overview: AdminUserServlet provides administrative governance over user accounts. Admins can view registered users, elevate customer roles to administrator or delivery staff, or remove accounts.

HTTP Methods & Flow Breakdown:

- **doGet():** Retrieves all registered user accounts from `UserDAO` and forwards to `userListAdmin.jsp`.
- **doPost():** Processes role elevation (`action=changeRole`) or user deletion (`action=delete`).

Key Implementation Highlights:

- **Role Escalation:** Enables dynamic permission updates (`customer`, `admin`, `delivery`).
- **User Governance:** Protects system integrity by allowing administrative account management.

Source Code (AdminUserServlet.java):

```

1 package com.app.controllers;
2
3 import com.app.dao.UserDAO;
4 import com.app.dao_implementation.UserDAOImpl;
5 import com.app.models.User;
6
7 import javax.servlet.ServletException;
8 import javax.servlet.annotation.WebServlet;
9 import javax.servlet.http.*;
10 import java.io.IOException;
11 import java.util.List;
12
13 @WebServlet("/admin/users")
14 public class AdminUserServlet extends HttpServlet {
15
16     private static final long serialVersionUID = 1L;
17     private UserDAO userDAO;
18
19     @Override
20     public void init() throws ServletException {
21         userDAO = new UserDAOImpl();
22     }
23
24     private boolean isAdmin(HttpServletRequest req) {
25         HttpSession session = req.getSession(false);
26         User u = (session != null) ? (User) session.getAttribute("loggedUser") : null;
27         return u != null && "admin".equalsIgnoreCase(u.getRole());
28     }
29
30     @Override
31     protected void doGet(HttpServletRequest req, HttpServletResponse resp)
32         throws ServletException, IOException {
33
34         if (!isAdmin(req)) {
35             resp.sendRedirect(req.getContextPath() + "/user?action=login");
36
37             return;
38         }
39
40         String action = req.getParameter("action");
41         if (action == null) action = "list";
42
43         switch (action) {
44             case "delete":
45                 int delUserId = Integer.parseInt(req.getParameter("userId"));
46                 userDAO.deleteUser(delUserId);
47                 resp.sendRedirect(req.getContextPath() + "/admin/users");
48                 break;
49
50             case "changeRole":
51                 int userId = Integer.parseInt(req.getParameter("userId"));
52                 String newRole = req.getParameter("role");
53                 if (newRole != null && !newRole.trim().isEmpty()) {
54                     User u = userDAO.getUser(userId);
55                     if (u != null) {
56                         u.setRole(newRole);
57                         userDAO.updateUser(u);
58                     }
59                 }
60                 resp.sendRedirect(req.getContextPath() + "/admin/users");
61                 break;
62
63             default: // list
64                 List<User> userList = userDAO.getAllUsers();
65                 req.setAttribute("users", userList);
66                 req.getRequestDispatcher("/jsp/admin/userListAdmin.jsp").forward(req, resp);
67                 break;
68         }
69
70     }
71
72     @Override

```

```
71     protected void doPost(HttpServletRequest req, HttpServletResponse resp)
72         throws ServletException, IOException {
73         doGet(req, resp);
74     }
75 }
76
```

11. AdminReviewServlet.java (URL: /admin/reviews)

Role / Access Level: Admin Only

Primary Purpose: Customer Feedback & Review Moderation Controller.

Architectural Overview: AdminReviewServlet allows administrators to monitor customer ratings and reviews across all Bangladeshi restaurants, enabling feedback moderation and deleting inappropriate reviews.

HTTP Methods & Flow Breakdown:

- **doGet():** Fetches all platform reviews via `ReviewDAO.getAllReviews()` and renders `reviewListAdmin.jsp`.
- **doPost():** Deletes specified review entries (`action=delete`) and triggers restaurant rating recalculations.

Key Implementation Highlights:

- **Quality Control:** Maintains review integrity across all platform restaurants.
- **Automatic Sync:** Recalculates restaurant star averages whenever a review is moderated.

Source Code (AdminReviewServlet.java):

```
1 package com.app.controllers;
2
3 import com.app.dao.RestaurantDAO;
4 import com.app.dao.ReviewDAO;
5 import com.app.dao.UserDAO;
6 import com.app.dao_implementation.RestaurantDAOImpl;
7 import com.app.dao_implementation.ReviewDAOImpl;
8 import com.app.dao_implementation.UserDAOImpl;
9 import com.app.models.Restaurant;
10 import com.app.models.Review;
11 import com.app.models.User;
12
13 import javax.servlet.ServletException;
14 import javax.servlet.annotation.WebServlet;
15 import javax.servlet.http.*;
16 import java.io.IOException;
17 import java.util.HashMap;
18 import java.util.List;
19 import java.util.Map;
20
21 @WebServlet("/admin/reviews")
22 public class AdminReviewServlet extends HttpServlet {
23
24     private static final long serialVersionUID = 1L;
25     private ReviewDAO reviewDAO;
26     private UserDAO userDAO;
27     private RestaurantDAO restaurantDAO;
28
29     @Override
30     public void init() throws ServletException {
31         reviewDAO = new ReviewDAOImpl();
32         userDAO = new UserDAOImpl();
33         restaurantDAO = new RestaurantDAOImpl();
34     }
35 }
```

```

36     private boolean isAdmin(HttpServletRequest req) {
37         HttpSession session = req.getSession(false);
38         User u = (session != null) ? (User) session.getAttribute("loggedUser") : null;
39         return u != null && "admin".equalsIgnoreCase(u.getRole());
40     }
41
42     @Override
43     protected void doGet(HttpServletRequest req, HttpServletResponse resp)
44         throws ServletException, IOException {
45
46         if (!isAdmin(req)) {
47             resp.sendRedirect(req.getContextPath() + "/user?action=login");
48             return;
49         }
50
51         String action = req.getParameter("action");
52         if (action == null) action = "list";
53
54         switch (action) {
55             case "delete":
56                 int delReviewId = Integer.parseInt(req.getParameter("reviewId"));
57                 reviewDAO.deleteReview(delReviewId);
58                 resp.sendRedirect(req.getContextPath() + "/admin/reviews");
59                 break;
60
61             default: // list
62                 List<Review> reviewList = reviewDAO.getAllReviews();
63                 Map<Integer, User> userMap = new HashMap<>();
64                 Map<Integer, Restaurant> restaurantMap = new HashMap<>();
65
66                 for (Review r : reviewList) {
67                     if (!userMap.containsKey(r.getUserId())) {
68                         userMap.put(r.getUserId(), userDAO.getUser(r.getUserId()));
69                     }
70                     if (!restaurantMap.containsKey(r.getRestaurantId())) {
71                         restaurantMap.put(r.getRestaurantId(),
72                             restaurantDAO.getRestaurantById(r.getRestaurantId()));
73                     }
74
75                     req.setAttribute("reviews", reviewList);
76                     req.setAttribute("userMap", userMap);
77                     req.setAttribute("restaurantMap", restaurantMap);
78                     req.getRequestDispatcher("/jsp/admin/reviewListAdmin.jsp").forward(req, resp);
79                     break;
80                 }
81             }
82
83     @Override
84     protected void doPost(HttpServletRequest req, HttpServletResponse resp)
85         throws ServletException, IOException {
86         doGet(req, resp);
87     }
88 }
89

```

12. AdminRestaurantServlet.java (URL: /admin/restaurants)

Role / Access Level: Admin Only

Primary Purpose: Bangladeshi Restaurant CRUD Management Controller.

Architectural Overview: AdminRestaurantServlet provides full administrative CRUD functionality for Bangladeshi restaurants. Admins can register new restaurants (name, cuisine, location, contact, image URL), edit profiles, or deactivate listings.

HTTP Methods & Flow Breakdown:

- **doGet():** Lists all restaurants for administrative management or renders edit forms.
- **doPost():** Handles adding (`action=add`), editing (`action=edit`), or deleting (`action=delete`) restaurant records

via `RestaurantDAO`.

Key Implementation Highlights:

- Profile Management: Controls restaurant visibility, address details in Dhaka, and image assets.
- Database Synchronization: Directly updates `restaurants` table in MySQL.

Source Code (AdminRestaurantServlet.java):

```
1 package com.app.controllers;
2
3 import com.app.dao.RestaurantDAO;
4 import com.app.dao_implementation.RestaurantDAOImpl;
5 import com.app.models.Restaurant;
6
7 import javax.servlet.ServletException;
8 import javax.servlet.annotation.WebServlet;
9 import javax.servlet.http.*;
10 import java.io.IOException;
11 import java.util.List;
12
13 @WebServlet("/admin/restaurants")
14 public class AdminRestaurantServlet extends HttpServlet {
15
16     /**
17      *
18      */
19     private static final long serialVersionUID = 1L;
20     private RestaurantDAO restaurantDAO;
21
22     @Override
23     public void init() throws ServletException {
24         restaurantDAO = new RestaurantDAOImpl();
25     }
26
27     // ■ Helper: Check admin role
28     private boolean isAdmin(HttpServletRequest req) {
29         HttpSession session = req.getSession(false);
30         return session != null && "admin".equals(session.getAttribute("role"));
31     }
32
33     @Override
34     protected void doGet(HttpServletRequest req, HttpServletResponse resp)
35         throws ServletException, IOException {
```

```

36
37     if (!isAdmin(req)) {
38         resp.sendRedirect(req.getContextPath() + "/unauthorized.jsp");
39         return;
40     }
41
42     String action = req.getParameter("action");
43
44     if (action == null) action = "list";
45
46     switch (action) {
47
48         case "add":
49             req.getRequestDispatcher("/jsp/admin/addRestaurant.jsp").forward(req, resp);
50             break;
51
52         case "edit":
53             int id = Integer.parseInt(req.getParameter("id"));
54             Restaurant r = restaurantDAO.getRestaurantById(id);
55
56             req.setAttribute("restaurant", r);
57             req.getRequestDispatcher("/jsp/admin/editRestaurant.jsp").forward(req, resp);
58             break;
59
60         case "delete":
61             int delId = Integer.parseInt(req.getParameter("id"));
62             restaurantDAO.deleteRestaurant(delId);
63             resp.sendRedirect(req.getContextPath() + "/admin/restaurants");
64             break;
65
66         default: // list
67             List<Restaurant> list = restaurantDAO.getAllRestaurants();
68             req.setAttribute("restaurants", list);
69             req.getRequestDispatcher("/jsp/admin/restaurantListAdmin.jsp").forward(req, resp);
70     }

```

```

71 }
72
73 @Override
74 protected void doPost(HttpServletRequest req, HttpServletResponse resp)
75     throws ServletException, IOException {
76
77     if (!isAdmin(req)) {
78         resp.sendRedirect(req.getContextPath() + "/unauthorized.jsp");
79         return;
80     }
81
82     String action = req.getParameter("action");
83
84     if ("save".equals(action)) {
85         Restaurant r = new Restaurant();
86         r.setName(req.getParameter("name"));
87         r.setDescription(req.getParameter("description"));
88         r.setCuisineType(req.getParameter("cuisineType"));
89         r.setAddress(req.getParameter("address"));
90         r.setCity(req.getParameter("city"));
91         r.setState(req.getParameter("state"));
92         r.setZip(req.getParameter("zip"));
93         r.setPhone(req.getParameter("phone"));
94         r.setEmail(req.getParameter("email"));
95         r.setRating(Double.parseDouble(req.getParameter("rating")));
96         r.setDeliveryTime(Integer.parseInt(req.getParameter("deliveryTime")));
97         r.setImageUrl(req.getParameter("imageUrl"));
98         r.setActive(Boolean.parseBoolean(req.getParameter("isActive")));
99
100         restaurantDAO.addRestaurant(r);
101         resp.sendRedirect(req.getContextPath() + "/admin/restaurants");
102     }
103
104     else if ("update".equals(action)) {
105         Restaurant r = new Restaurant();

```

```

106         r.setRestaurantId(Integer.parseInt(req.getParameter("restaurantId")));
107         r.setName(req.getParameter("name"));
108         r.setDescription(req.getParameter("description"));
109         r.setCuisineType(req.getParameter("cuisineType"));
110         r.setAddress(req.getParameter("address"));
111         r.setCity(req.getParameter("city"));
112         r.setState(req.getParameter("state"));
113         r.setZip(req.getParameter("zip"));
114         r.setPhone(req.getParameter("phone"));
115         r.setEmail(req.getParameter("email"));
116         r.setRating(Double.parseDouble(req.getParameter("rating")));
117         r.setDeliveryTime(Integer.parseInt(req.getParameter("deliveryTime")));
118         r.setImageUrl(req.getParameter("imageUrl"));
119         r.setActive(Boolean.parseBoolean(req.getParameter("isActive")));
120
121         restaurantDAO.updateRestaurant(r);
122         resp.sendRedirect(req.getContextPath() + "/admin/restaurants");
123     }
124 }
125 }
126
127 /*
128 =====
129 AdminRestaurantServlet - JSP Mappings (For Future Reference)
130 =====
131
132 URL MAPPINGS:
133 -----
134 GET /admin/restaurants?action=list
135     → /WEB-INF/jsp/admin/listRestaurants.jsp
136
137 GET /admin/restaurants?action=add
138     → /WEB-INF/jsp/admin/addRestaurant.jsp
139
140 GET /admin/restaurants?action=edit&id={id}
141     → /WEB-INF/jsp/admin/editRestaurant.jsp
142
143 GET /admin/restaurants?action=delete&id={id}
144     → (No JSP – Performs delete, then redirects to list)
145
146 POST /admin/restaurants?action=save
147     → After saving → redirect to /admin/restaurants
148
149 POST /admin/restaurants?action=update
150     → After updating → redirect to /admin/restaurants
151
152
153 REQUIRED JSP FILES (YOU MUST CREATE THESE):
154 -----
155 ■ /WEB-INF/jsp/admin/listRestaurants.jsp
156 ■ /WEB-INF/jsp/admin/addRestaurant.jsp
157 ■ /WEB-INF/jsp/admin/editRestaurant.jsp
158
159 */
160 */
161

```

13. AdminMenuServlet.java (URL: /admin/menu)

Role / Access Level: Admin Only

Primary Purpose: Restaurant Menu Items & Dish Pricing (■) CRUD Controller.

Architectural Overview: AdminMenuServlet gives admins control over dish catalogs for each restaurant. Admins can add new dishes (Kacchi Biryani, Morog Polao, Grill Chicken), set prices in Taka (■), update descriptions, and toggle dish availability.

HTTP Methods & Flow Breakdown:

- **doGet():** Fetches menu items for a specific restaurant ID or loads dish edit form.

- **doPost()**: Processes dish creation (`action=add`), updating (`action=edit`), deletion (`action=delete`), or availability toggling (`action=toggleAvailability`).

Key Implementation Highlights:

- Currency Management: Persists dish pricing in Bangladeshi Taka (₳).
- Availability Toggling: Enables instant stock status updates for menu items.

Source Code (AdminMenuServlet.java):

```
1 package com.app.controllers;
2
3 import java.io.IOException;
4 import java.util.List;
5 import javax.servlet.*;
6 import javax.servlet.annotation.WebServlet;
7 import javax.servlet.http.*;
8
9 import com.app.dao.MenuItemDAO;
10 import com.app.dao_implementation.MenuItemDAOImpl;
11 import com.app.models.MenuItem;
12 import com.app.dao.RestaurantDAO;
13 import com.app.dao_implementation.RestaurantDAOImpl;
14 import com.app.models.Restaurant;
15
16 @WebServlet("/admin/menu")
17 public class AdminMenuServlet extends HttpServlet {
18
19     /**
20     *
21     */
22     private static final long serialVersionUID = 1L;
23     private final MenuItemDAO menuItemDAO = new MenuItemDAOImpl();
24     private final RestaurantDAO restaurantDAO = new RestaurantDAOImpl();
25
26     @Override
27     protected void doGet(HttpServletRequest request, HttpServletResponse response)
28         throws ServletException, IOException {
29
30         String action = request.getParameter("action");
31         if (action == null) action = "list";
32
33         switch (action) {
34
35             case "add": // Show Add Form
```

```

36         showAddForm(request, response);
37         break;
38
39         case "edit": // Show Edit Form
40             showEditForm(request, response);
41             break;
42
43         case "delete": // Delete menu item
44             deleteMenuItem(request, response);
45             break;
46
47         default: // List all menu items for a restaurant
48             listMenuItems(request, response);
49             break;
50     }
51 }
52
53 // ----- LIST MENU ITEMS -----
54 private void listMenuItems(HttpServletRequest request, HttpServletResponse response)
55     throws ServletException, IOException {
56
57     String resIdParam = request.getParameter("restaurantId");
58     if (resIdParam == null || resIdParam.trim().isEmpty()) {
59         List<Restaurant> restaurants = restaurantDAO.getAllRestaurants();
60         request.setAttribute("restaurants", restaurants);
61         RequestDispatcher rd = request.getRequestDispatcher(
62             "/jsp/admin/selectRestaurantMenu.jsp");
63         rd.forward(request, response);
64         return;
65     }
66     int restaurantId = Integer.parseInt(resIdParam);
67     List<MenuItem> menuItems = menuItemDAO.getMenuItemsByRestaurantId(restaurantId);
68     Restaurant restaurant = restaurantDAO.getRestaurantById(restaurantId);
69
70     request.setAttribute("restaurantId", restaurantId);
71
72     request.setAttribute("restaurant", restaurant);
73     request.setAttribute("menuItems", menuItems);
74
75     RequestDispatcher rd = request.getRequestDispatcher(
76         "/jsp/admin/menuListAdmin.jsp");
77     rd.forward(request, response);
78 }
79
80 // ----- SHOW ADD FORM -----
81 private void showAddForm(HttpServletRequest request, HttpServletResponse response)
82     throws ServletException, IOException {
83
84     String resIdParam = request.getParameter("restaurantId");
85     if (resIdParam == null || resIdParam.trim().isEmpty()) {
86         response.sendRedirect(request.getContextPath() + "/admin/restaurants");
87         return;
88     }
89     int restaurantId = Integer.parseInt(resIdParam);
90     request.setAttribute("restaurantId", restaurantId);
91
92     RequestDispatcher rd = request.getRequestDispatcher(
93         "/jsp/admin/addMenuItem.jsp");
94     rd.forward(request, response);
95 }
96
97 // ----- SHOW EDIT FORM -----
98 private void showEditForm(HttpServletRequest request, HttpServletResponse response)
99     throws ServletException, IOException {
100
101     String idParam = request.getParameter("id");
102     if (idParam == null || idParam.trim().isEmpty()) {
103         response.sendRedirect(request.getContextPath() + "/admin/restaurants");
104         return;
105     }
106     int menuItemId = Integer.parseInt(idParam);

```

```

106     MenuItem menuItem = menuItemDAO.getMenuItemById(menuItemId);
107
108     request.setAttribute("menuItem", menuItem);
109
110     RequestDispatcher rd = request.getRequestDispatcher(
111         "/jsp/admin/editMenuItem.jsp");
112     rd.forward(request, response);
113 }
114
115 // ----- DELETE MENU ITEM -----
116 private void deleteMenuItem(HttpServletRequest request, HttpServletResponse response)
117     throws IOException {
118
119     int menuItemId = Integer.parseInt(request.getParameter("id"));
120     MenuItem item = menuItemDAO.getMenuItemById(menuItemId);
121
122     int restaurantId = item.getRestaurantId();
123
124     menuItemDAO.deleteMenuItem(menuItemId);
125
126     response.sendRedirect(
127         request.getContextPath() + "/admin/menu?action=list&restaurantId=" + restaurantId);
128 }
129
130 // ----- POST (SAVE/UPDATE) -----
131 @Override
132 protected void doPost(HttpServletRequest request, HttpServletResponse response)
133     throws ServletException, IOException {
134
135     String action = request.getParameter("action");
136
137     if ("save".equals(action)) {
138         saveMenuItem(request, response);
139     } else if ("update".equals(action)) {
140         updateMenuItem(request, response);
141     }
142 }
143
144 // ----- SAVE NEW MENU ITEM -----
145 private void saveMenuItem(HttpServletRequest request, HttpServletResponse response)
146     throws IOException {
147
148     MenuItem menuItem = new MenuItem();
149
150     menuItem.setRestaurantId(Integer.parseInt(request.getParameter("restaurantId")));
151     menuItem.setName(request.getParameter("name"));
152     menuItem.setDescription(request.getParameter("description"));
153     menuItem.setPrice(Double.parseDouble(request.getParameter("price")));
154     menuItem.setCategory(request.getParameter("category"));
155     menuItem.setVeg("veg".equalsIgnoreCase(request.getParameter("isVeg")));
156     menuItem.setAvailable("available".equalsIgnoreCase(request.getParameter("isAvailable")));
157     menuItem.setImageUrl(request.getParameter("imageUrl"));
158
159     menuItemDAO.addMenuItem(menuItem);
160
161     response.sendRedirect(
162         request.getContextPath() + "/admin/menu?action=list&restaurantId=" +
menuItem.getRestaurantId());
163 }
164
165 // ----- UPDATE MENU ITEM -----
166 private void updateMenuItem(HttpServletRequest request, HttpServletResponse response)
167     throws IOException {
168
169     MenuItem menuItem = new MenuItem();
170
171     menuItem.setMenuItemId(Integer.parseInt(request.getParameter("menuItemId")));
172     menuItem.setRestaurantId(Integer.parseInt(request.getParameter("restaurantId")));
173     menuItem.setName(request.getParameter("name"));
174     menuItem.setDescription(request.getParameter("description"));
175     menuItem.setPrice(Double.parseDouble(request.getParameter("price")));

```

```

176         menuItem.setCategory(request.getParameter("category"));
177         menuItem.setVeg("veg".equalsIgnoreCase(request.getParameter("isVeg")));
178         menuItem.setAvailable("available".equalsIgnoreCase(request.getParameter("isAvailable")));
179         menuItem.setImageUrl(request.getParameter("imageUrl"));
180
181         menuItemDAO.updateMenuItem(menuItem);
182
183         response.sendRedirect(
184             request.getContextPath() + "/admin/menu?action=list&restaurantId=" +
menuItem.getRestaurantId());
185     }
186 }
187
188
189 /*
190 =====
191 AdminMenuServlet - JSP Mappings (For Future Reference)
192 =====
193
194 URL MAPPINGS:
195 -----
196 GET /admin/menu?action=list&restaurantId={id}
197   → /WEB-INF/jsp/admin/menu/listMenuItems.jsp
198
199 GET /admin/menu?action=add&restaurantId={id}
200   → /WEB-INF/jsp/admin/menu/addMenuItem.jsp
201
202 GET /admin/menu?action=edit&id={menuItemId}
203   → /WEB-INF/jsp/admin/menu/editMenuItem.jsp
204
205 GET /admin/menu?action=delete&id={menuItemId}
206   → (No JSP – Performs delete, then redirects to list)
207
208 POST /admin/menu?action=save
209   → After saving → redirect to /admin/menu?action=list&restaurantId={id}
210
211 POST /admin/menu?action=update
212   → After updating → redirect to /admin/menu?action=list&restaurantId={id}
213
214
215 REQUIRED JSP FILES (YOU MUST CREATE THESE):
216 -----
217 ■ /WEB-INF/jsp/admin/menu/listMenuItems.jsp
218 ■ /WEB-INF/jsp/admin/menu/addMenuItem.jsp
219 ■ /WEB-INF/jsp/admin/menu/editMenuItem.jsp
220
221
222
223 */

```

14. MenuServlet.java (URL: /menu)

Role / Access Level: Public / Customer

Primary Purpose: Menu Item Query & Search Controller.

Architectural Overview: MenuServlet provides customer-facing menu item lookup and search queries across dishes and cuisines.

HTTP Methods & Flow Breakdown:

- **doGet():** Queries dishes by restaurant ID or search keyword using `MenuDAO` and forwards results to `menuList.jsp`.
- **doPost():** Delegates search submissions to `doGet()`.

Key Implementation Highlights:

- **Filtered Lookup:** Supports searching dishes by name, category, or restaurant.
- **Real-Time Display:** Attaches dish collections to `\${menuItems}` for JSP rendering.

Source Code (MenuServlet.java):

```

1 package com.app.controllers;
2
3 import java.io.IOException;
4 import java.util.List;
5 import javax.servlet.*;
6 import javax.servlet.annotation.WebServlet;
7 import javax.servlet.http.*;
8
9 import com.app.dao.MenuItemDAO;
10 import com.app.dao.RestaurantDAO;
11 import com.app.dao_implementation.MenuItemDAOImpl;
12 import com.app.dao_implementation.RestaurantDAOImpl;
13 import com.app.models.MenuItem;
14 import com.app.models.Restaurant;
15
16
17 @WebServlet("/menu")
18 public class MenuServlet extends HttpServlet {
19
20     /**
21     *
22     */
23     private static final long serialVersionUID = 1L;
24     private MenuItemDAO menuItemDAO = new MenuItemDAOImpl();
25     private RestaurantDAO restaurantDAO = new RestaurantDAOImpl();
26
27     @Override
28     protected void doGet(HttpServletRequest request, HttpServletResponse response)
29         throws ServletException, IOException {
30
31         // Validate restaurantId
32         String idParam = request.getParameter("restaurantId");
33         if (idParam == null) {
34             response.sendError(HttpServletResponse.SC_BAD_REQUEST, "Missing restaurantId");
35             return;
36         }
37
38         int restaurantId = Integer.parseInt(idParam);
39
40         // Fetch restaurant details
41         Restaurant restaurant = restaurantDAO.getRestaurantById(restaurantId);
42         if (restaurant == null || !restaurant.isActive()) {
43             response.sendError(HttpServletResponse.SC_NOT_FOUND, "Restaurant not found");
44             return;
45         }
46
47         // Fetch menu items for this restaurant
48         List<MenuItem> menuItems = menuItemDAO.getMenuItemsByRestaurantId(restaurantId);
49
50         // Pass data to JSP
51         request.setAttribute("restaurant", restaurant);
52         request.setAttribute("menuItems", menuItems);
53
54         RequestDispatcher rd = request.getRequestDispatcher(
55             "/WEB-INF/jsp/customer/menu.jsp");
56         rd.forward(request, response);
57     }
58 }
59
60 /*
61 =====
62 MenuServlet - JSP Mappings (Customer-Facing)
63 =====
64
65 CUSTOMER URL:
66 -----
67 GET /menu?restaurantId={id}
68     → /WEB-INF/jsp/customer/menu.jsp
69
70

```

```

71 REQUIRED JSP FILES:
72 -----
73 ■ /WEB-INF/jsp/customer/menu.jsp
74
75
76 DATA PASSED TO JSP:
77 -----
78 restaurant → Restaurant object (name, rating, etc.)
79 menuItems → List<MenuItem> for that restaurant
80
81
82 NOTES:
83 -----
84 ✓ This servlet is public → no login required.
85 ✓ Only fetches & displays data (NO CRUD).
86 ✓ Uses MenuItemDAO → getMenuItemsByRestaurantId()
87 ✓ Uses RestaurantDAO → getRestaurantById()
88 ✓ MenuItem POJO fields correctly mapped:
89     name, description, price, isVeg, isAvailable, imageUrl
90 =====
91 */
92
93

```

15. OrderSummaryServlet.java (URL: /orderSummary)

Role / Access Level: Customer

Primary Purpose: Pre-Checkout Itemized Invoice Review Controller.

Architectural Overview: OrderSummaryServlet renders a pre-checkout itemized invoice breakdown, displaying selected items, subtotal, tax, delivery fees, and total in Taka (₳).

HTTP Methods & Flow Breakdown:

- **doGet():** Reads active session cart, computes tax/delivery fees, attaches summary details to request scope, and forwards to `orderSummary.jsp`.
- **doPost():** Delegates to `doGet()`.

Key Implementation Highlights:

- **Invoice Verification:** Allows customers to verify order breakdown before choosing address and payment method.

Source Code (OrderSummaryServlet.java):

```
1 package com.app.controllers;
2
3 import com.app.dao.*;
4 import com.app.dao_implementation.*;
5 import com.app.models.*;
6
7 import javax.servlet.ServletException;
8 import javax.servlet.annotation.WebServlet;
9 import javax.servlet.http.*;
10 import java.io.IOException;
11 import java.util.*;
12
13 @WebServlet("/orderSummary")
14 public class OrderSummaryServlet extends HttpServlet {
15
16     private final OrderDAO orderDAO = new OrderDAOImpl();
17     private final RestaurantDAO restaurantDAO = new RestaurantDAOImpl();
18     private final MenuItemDAO menuItemDAO = new MenuItemDAOImpl();
19     private final AddressDAO addressDAO = new AddressDAOImpl();
20
21     @Override
22     protected void doGet(HttpServletRequest req, HttpServletResponse resp)
23         throws ServletException, IOException {
24
25         HttpSession session = req.getSession(false);
26         com.app.models.User loggedInUser = (session != null) ? (com.app.models.User)
27 session.getAttribute("loggedInUser") : null;
28         if (loggedInUser == null) {
29             resp.sendRedirect(req.getContextPath() + "/user?action=login");
30             return;
31         }
32         int userId = loggedInUser.getUserId();
33
34         String orderIdParam = req.getParameter("orderId");
35         if (orderIdParam == null) {
36             resp.sendRedirect(req.getContextPath() + "/orderHistory");
37
38             return;
39         }
40
41         int orderId = Integer.parseInt(orderIdParam);
42
43         Order order = orderDAO.getOrderById(orderId);
44         if (order == null || order.getUserId() != userId) {
45             resp.sendRedirect(req.getContextPath() + "/orderHistory");
46             return;
47         }
48
49         List<OrderItem> items = orderDAO.getOrderItems(orderId);
50
51         // Fetch Restaurant details
52         Restaurant restaurant = restaurantDAO.getRestaurantById(order.getRestaurantId());
53
54         // Fetch MenuItems details
55         Set<Integer> menuIds = new HashSet<>();
56         for (OrderItem item : items) {
57             menuIds.add(item.getMenuItemId());
58         }
59         Map<Integer, MenuItem> menuItemMap = menuItemDAO.getMenuItemsAsMap(menuIds);
60
61         // Fetch Address details
62         Address address = null;
63         if (order.getAddressId() != null) {
64             address = addressDAO.getAddressById(order.getAddressId(), userId);
65         }
66
67         req.setAttribute("order", order);
68         req.setAttribute("items", items);
69         req.setAttribute("restaurant", restaurant);
70         req.setAttribute("menuItemMap", menuItemMap);
71         req.setAttribute("address", address);
72     }
73 }
```

```
71         req.getRequestDispatcher("/jsp/customer/order_summary.jsp")
72         .forward(req, resp);
73     }
74 }
75
```

16. PaymentServlet.java (URL: /payment)

Role / Access Level: Customer

Primary Purpose: Payment Processing & Gateway Simulation Controller.

Architectural Overview: PaymentServlet simulates payment processing for bKash, Nagad, Credit/Debit Cards, and Cash on Delivery (COD).

HTTP Methods & Flow Breakdown:

- **doGet():** Renders payment selection interface (`payment.jsp`).
- **doPost():** Validates payment details, updates order payment status to `PAID` or `PENDING\_COD`, and forwards to order confirmation.

Key Implementation Highlights:

- **Multi-Channel Support:** Handles Bangladeshi mobile financial services (bKash/Nagad) alongside traditional COD.

Source Code (PaymentServlet.java):

```
1 package com.app.controllers;
2
3 import com.app.dao.OrderDAO;
4 import com.app.dao_implementation.OrderDAOImpl;
5
6 import javax.servlet.ServletException;
7 import javax.servlet.annotation.WebServlet;
8 import javax.servlet.http.*;
9 import java.io.IOException;
10
11 @WebServlet("/payment")
12 public class PaymentServlet extends HttpServlet {
13
14     private final OrderDAO orderDAO = new OrderDAOImpl();
15
16     @Override
17     protected void doGet(HttpServletRequest req, HttpServletResponse resp)
18         throws ServletException, IOException {
19
20         // Ensure user logged in
21         HttpSession session = req.getSession(false);
22         com.app.models.User loggedInUser = (session != null) ? (com.app.models.User)
23 session.getAttribute("loggedInUser") : null;
24         if (loggedInUser == null) {
25             resp.sendRedirect(req.getContextPath() + "/user?action=login");
26             return;
27         }
28         int userId = loggedInUser.getUserId();
29
30         String orderIdParam = req.getParameter("orderId");
31         if (orderIdParam == null) {
32             resp.sendRedirect(req.getContextPath() + "/restaurant");
33             return;
34         }
35         req.setAttribute("orderId", orderIdParam);
36     }
37 }
```

```
36
37     req.getRequestDispatcher("/jsp/customer/payment.jsp")
38         .forward(req, resp);
39 }
40
41 @Override
42 protected void doPost(HttpServletRequest req, HttpServletResponse resp)
43     throws ServletException, IOException {
44
45     HttpSession session = req.getSession(false);
46     com.app.models.User loggedUser = (session != null) ? (com.app.models.User)
session.getAttribute("loggedUser") : null;
47     if (loggedUser == null) {
48         resp.sendRedirect(req.getContextPath() + "/user?action=login");
49         return;
50     }
51     int userId = loggedUser.getUserId();
52
53     String orderIdParam = req.getParameter("orderId");
54     if (orderIdParam == null) {
55         resp.sendRedirect(req.getContextPath() + "/restaurant");
56         return;
57     }
58
59     int orderId = Integer.parseInt(orderIdParam);
60
61     // Your DB supports: pending → completed
62     orderDAO.updatePaymentStatus(orderId, "completed");
63
64     // After completing payment → restaurant accepts automatically
65     orderDAO.updateOrderStatus(orderId, "accepted");
66
67     resp.sendRedirect(req.getContextPath() + "/orderSummary?orderId=" + orderId);
68 }
69 }
70
```

17. OrderSuccessServlet.java (URL: /orderSuccess)

Role / Access Level: Customer

Primary Purpose: Order Confirmation & Final Receipt Controller.

Architectural Overview: OrderSuccessServlet displays the final order confirmation receipt after successful checkout.

HTTP Methods & Flow Breakdown:

- **doGet():** Accepts `orderId` parameter, fetches order details via `OrderDAO.getOrderById()`, and renders `orderSuccess.jsp`.
- **doPost():** Delegates to `doGet()`.

Key Implementation Highlights:

- **Receipt Generation:** Displays order ID, estimated delivery time, and final bill breakdown in Taka (₳).

Source Code (OrderSuccessServlet.java):

```
1 package com.app.controllers;
2
3 import javax.servlet.*;
4 import javax.servlet.annotation.WebServlet;
5 import javax.servlet.http.*;
6 import java.io.IOException;
7
8 @WebServlet("/orderSuccess")
9 public class OrderSuccessServlet extends HttpServlet {
10
11     @Override
12     protected void doGet(HttpServletRequest req, HttpServletResponse resp)
13         throws ServletException, IOException {
14
15         req.getRequestDispatcher("/jsp/customer/orderSuccess.jsp")
16             .forward(req, resp);
17     }
18 }
19
```